

WerkstattPilot

Digitale Auftragsabwicklung für KFZ-Werkstätten

WerkstattPilot

Digitale Auftragsabwicklung für KFZ-Werkstätten



Projektarbeit

Software Engineering 1
Arbeitsgruppe 3

Team

Livio Luna
Dominic Lolos

Abgabe

10.06.2026
TS TSI 2502 A

Ausgangslage und Problemstellung

Warum WerkstattPilot benötigt wird

Medienbrüche

Aufträge, Arbeitszeiten und Teile werden heute oft mit Papier und Excel erfasst.

Doppelte Erfassung

Manuelle Übertragung verursacht Zeitverlust und erhöht das Fehlerrisiko.

Fehlender Überblick

Status, Termine und Lagerbestand sind nicht jederzeit transparent.

Ziel ist eine lokale, einfache und fehlerarme Digitalisierung der Auftragsabwicklung.

Die 1. Iteration konzentriert sich auf UC1 Auftrag anlegen und UC2 Auftragsübersicht mit Statuswechsel.

Ziele und Scope der 1. Iteration

Fokus auf hohen Nutzen und realistischen Implementierungsumfang

01 Auftragserfassung

Kunde, Fahrzeug,
Beanstandung und Termin
einheitlich erfassen.

02 Transparenz

Aufträge anzeigen und
Status nachvollziehbar
ändern.

03 Lagerführung

Teileverbrauch später
nachvollziehbar machen.

04 Dokumentation

Belege lokal als PDF oder
Druckausgabe vorbereiten.

Priorisiert wurden UC1 und UC2, da sie direkten Nutzen für Annahme, Mechanik und Serviceleitung bringen.

Aufgabe 5 ist bewusst ein kleiner Java-Prototyp: ConsoleClient, Mock-DB und harte Testdaten.

Zielbild und Prototyp sind getrennt: spätere Iterationen könnten GUI, Lagerbestand und PDF-Belege ergänzen.

Projektumgebung

Gemeinsame Struktur, Werkzeuge und Zusammenarbeit

Ordnerstruktur: Installation, Dokumentation, Entwicklung, Issue Tracking, Versionen und Abgabe.

Werkzeuge: Eclipse IDE, JDK 21, Git/GitHub Desktop, Visual Paradigm, Word und Excel.

Schreibweise: Java-Klassen in PascalCase, Methoden/Attribute in camelCase, Packages klein.

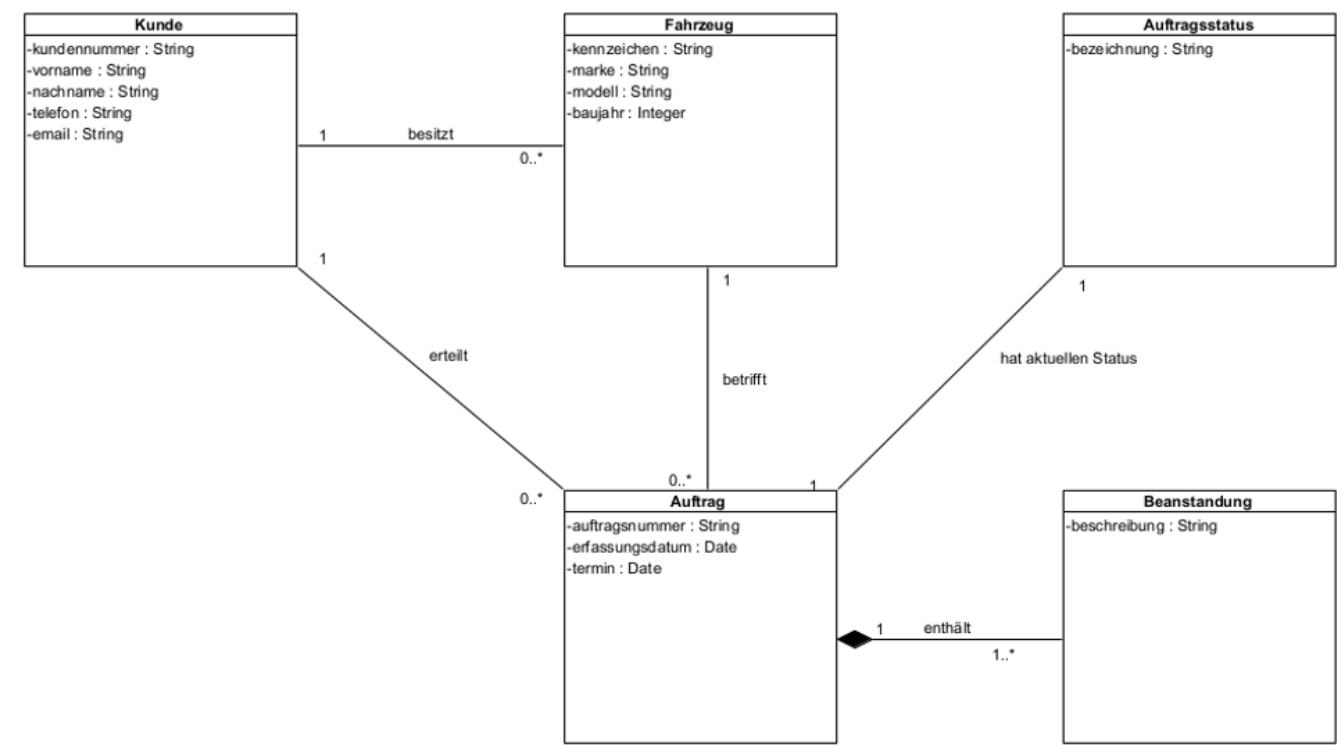
Beide Teammitglieder haben alle Aufgaben reviewed; Schwerpunkte dienen nur der Koordination.

Roter Faden

Projektauftrag -> Projektumgebung -> Analyse -> Design -> Implementation -> Testing -> Abgabecheck

Objektorientierte Analyse

Fachklassenmodell für UC1 und UC2



Zentrale Klasse: Auftrag.

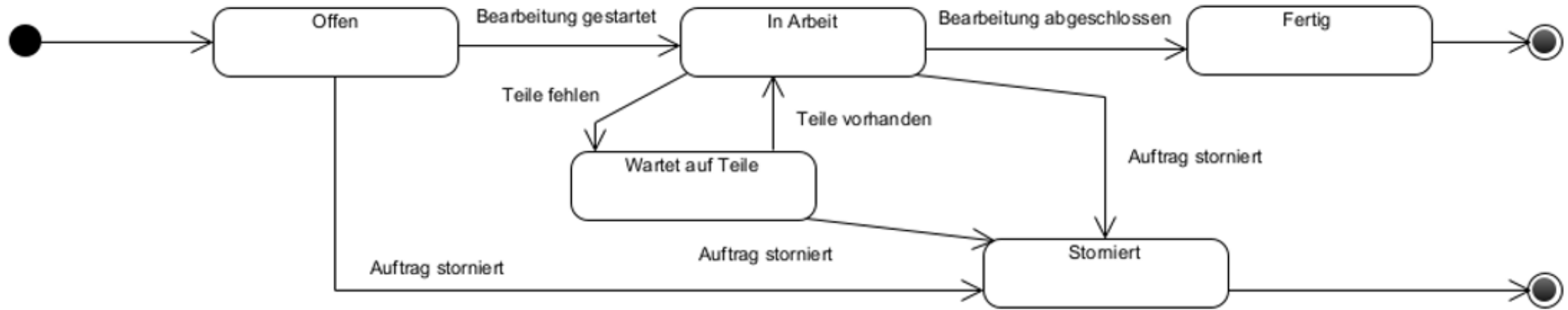
Auftrag verbindet Kunde, Fahrzeug, Beanstandung und Status.

Vererbung wurde geprüft, aber für Iteration 1 nicht eingesetzt.

Das Modell bleibt fachlich und verwendet Kundensprache.

Zustandsmodell Auftrag

Lebenszyklus eines Werkstattauftrags



OFFEN

IN_ARBEIT

WARTET_AUF_TEILE

FERTIG

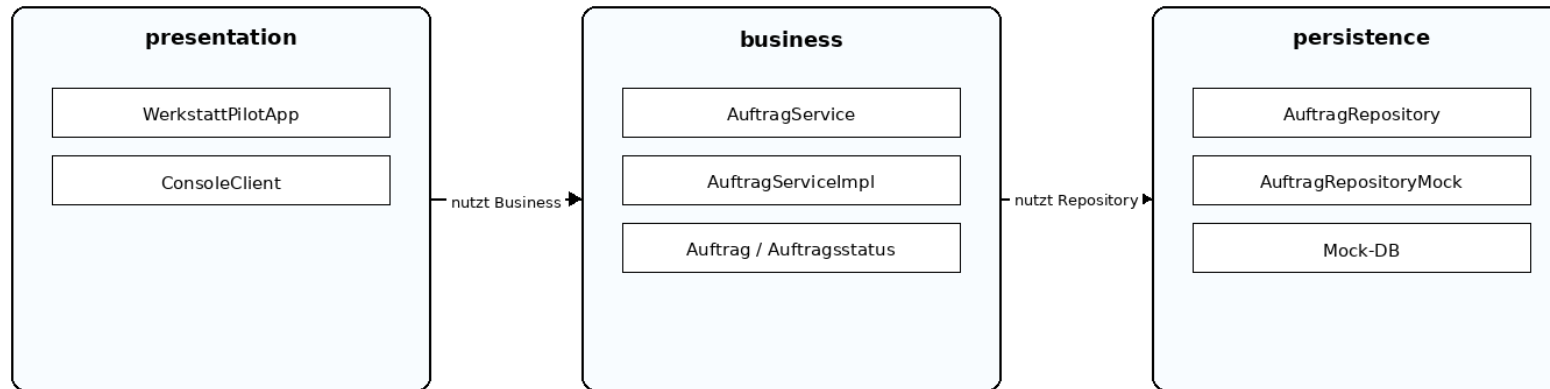
STORNIERT

Statuswerte wurden mit dem Java-Enum abgeglichen.

Objektorientiertes Design

3-Schichtenarchitektur und technische Abhängigkeiten

UML Komponentendiagramm: 3-Schichtenarchitektur



Regel: Die Presentation-Schicht greift nicht direkt auf die Persistence-Schicht zu.

Presentation: WerkstattPilotApp und ConsoleClient.

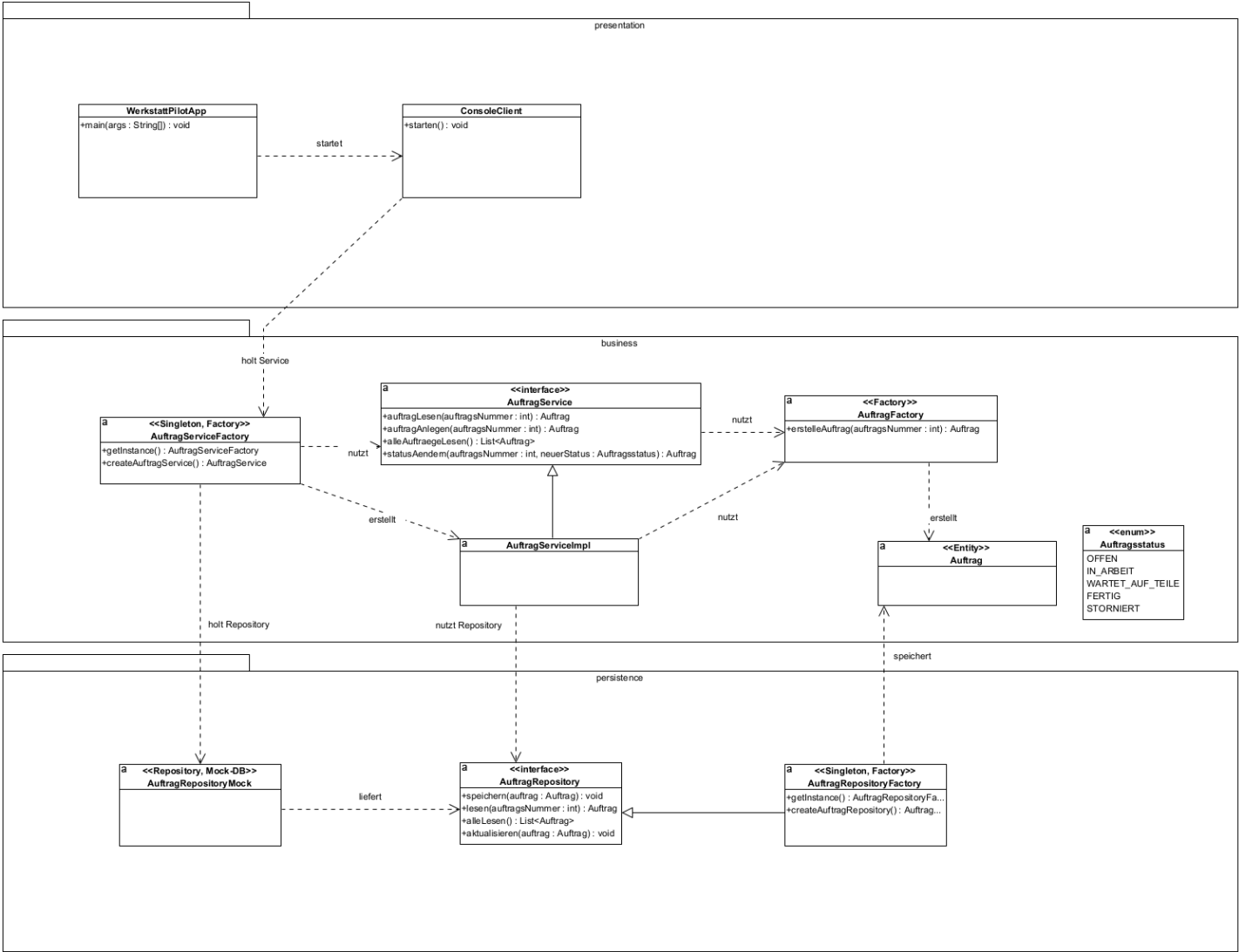
Business: AuftragService, Auftrag, Auftragsstatus und Factories.

Persistence: AuftragRepository und Mock-DB.

Wichtig: Presentation greift nicht direkt auf Persistence zu.

Designmodell

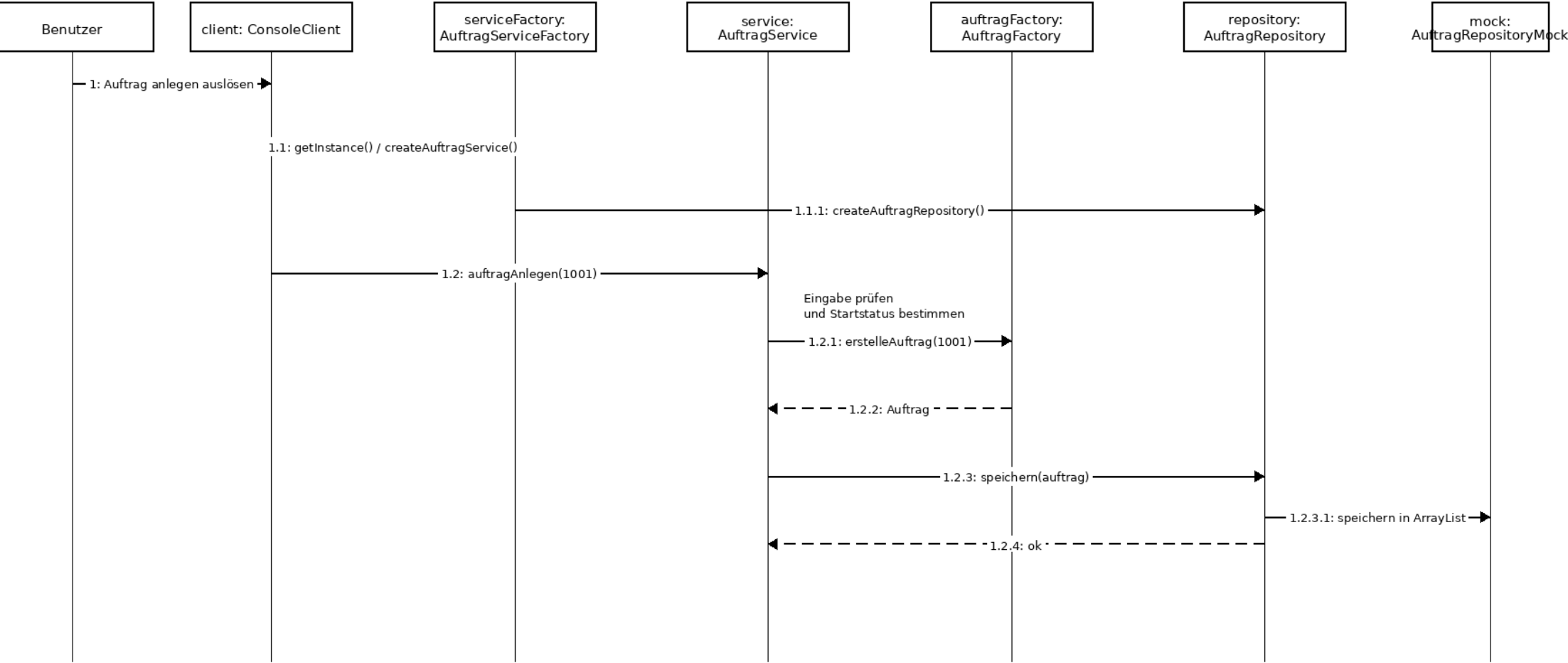
Repository, Factory und Singleton sichtbar eingesetzt



Sequenzdiagramm UC1 Auftrag anlegen

Normalablauf vom ConsoleClient bis zur Mock-DB

UML Sequenzdiagramm: UC1 Auftrag anlegen



Implementation in Java

Basic-Java-Prototyp ohne externe Datenbank

Package presentation: WerkstattPilotApp, ConsoleClient

Package business: Auftrag, AuftragService, AuftragServiceImpl, AuftragFactory, Auftragsstatus

Package persistence: AuftragRepository, AuftragRepositoryFactory, AuftragRepositoryMock

Test-Packages: einfache Java-Testklassen ohne externe Bibliothek

Mock-DB im Hauptspeicher mit ArrayList

Implementierte fachliche Klasse

Auftrag

Attribute im Prototyp:

- auftragsNummer
- status

Weitere fachliche Attribute werden im Zielbild beschrieben, aber nicht im kleinen Prototyp umgesetzt.

Testing und Endbenutzertest

Nachweis der wichtigsten Funktionen

Persistence-Test

speichern, lesen, alleLesen und aktualisieren wurden erfolgreich getestet.

Business-Test

auftragAnlegen, auftragLesen, alleAuftraegeLesen und statusAendern wurden erfolgreich getestet.

Endbenutzertest

ConsoleClient erstellt mehrere Aufträge und zeigt Statuswechsel auf der Konsole an.

Alle Persistence-Tests erfolgreich.
Alle Business-Tests erfolgreich.
WerkstattPilot Endbenutzertest: 3 Aufträge erstellt und Status geändert.

Javadoc und Abgabe

Finaler technischer Stand

Java-Quelltexte enthalten Dateiköpfe und einfache Javadoc-Kommentare.

Javadoc wurde erzeugt und im Ordner javadoc_html abgelegt.

Eclipse-Projekt wurde als ZIP für die Abgabe vorbereitet.

Projektdokument und Arbeitsprotokoll wurden final bereinigt.

Abgabedateien

arbeitsgruppe3-projektarbeit.pdf

arbeitsgruppe3-praesentation.pdf

arbeitsgruppe3-eclipse-projekt.zip

arbeitsgruppe3-arbeitsprotokoll.pdf

Fazit

WerkstattPilot als durchgängiger Software-Engineering-Prozess

Der rote Faden ist durchgängig: Auftrag und Status stehen im Zentrum.

Analyse, Design und Code wurden auf UC1 und UC2 ausgerichtet.

Die 3-Schichtenarchitektur trennt Bedienung, Fachlogik und Datenzugriff.

Der Prototyp zeigt die wichtigsten Funktionen in einfacher, prüfbarer Form.